Manual do ecossistema de desenvolvimento

> Para quem nunca usou. Ensina o que é cada peça, quando ela entra e o que digitar.
> Se você só quer os comandos, pule para "Cheat sheet".

1. A ideia em um minuto

O ecossistema responde duas perguntas diferentes, e por isso tem duas camadas:

- **O que vamos construir, por quê, e como isso é comunicado e entregue?** — skill `iniciar-projeto`.
- **Este código faz mesmo o que foi pedido, e como sabemos disso?** — skill `fuzkrigo-spec-driven`.

A primeira produz documentos e organiza o trabalho. A segunda produz **prova**: cada requisito vira um teste anotado, o teste roda de verdade, e um comando decide se a feature está pronta. "Pronto" deixa de ser opinião e passa a ser código de saída zero.

Nada aqui impede você de trabalhar rápido. O que ele impede é você **achar** que terminou.

2. As cinco peças

Peça	O que é	Quem dispara
`CLAUDE.md`	As regras que valem em toda sessão: persona, idioma, TDD, anti-alucinação, proibições	Carregado automaticamente, sempre
`references/`	Padrões consultáveis: convenções de Git, templates de issue e PR	Lido quando o assunto aparece
`skills/`	Fluxos completos: abertura de projeto, desenvolvimento de feature, ambiente corporativo	Você chama (`/iniciar-projeto`) ou o gatilho da descrição dispara

Peça	O que é	Quem dispara
`agents/`	Especialistas que rodam isolados e devolvem um relatório: revisão de código, segurança, documentação, board, docstrings	Chamados em pontos fixos do fluxo, ou quando você pede
`hooks/`	Dois scripts que o harness executa antes de cada escrita de arquivo	Automático, você não controla

Sobre os agentes: eles rodam em uma janela de contexto separada e **não veem a conversa**. Por isso devolvem relatórios completos em vez de conversar. Dois deles não têm permissão de escrita de propósito — `code-reviewer` e `security-reviewer` só leem e apontam.

Sobre os hooks, que são a parte que ninguém pode desligar sem querer:

- `check_planning_docs.py` — antes de escrever código de produção, verifica se existe planejamento (`docs/ROADMAP.md + docs/CHECKLIST.md` , **ou** uma spec em `.spec/features/`). Se não houver, ele **pergunta** (não bloqueia): quem decide é você.
- `check_style_prohibitions.py` — bloqueia emoji e frases que atribuem autoria do trabalho a uma ferramenta. Esse **bloqueia mesmo**, e é intencional. Tem uma exceção por caminho: dentro de `~/.claude/` e do repositório de backup, citar as frases proibidas é permitido, porque é onde a regra precisa ser documentada. Emoji continua bloqueado em todo lugar.

Não confunda com as ferramentas que já vêm no Claude Code

Além do que é seu, o produto traz skills próprias, acionadas por barra. Duas têm nome parecido com os seus agentes, e fazem coisas diferentes:

Você digita	O que roda	Não confundir com
`/code-review`	Skill do produto: revisa o diff atual, um PR ou uma branch, com critérios genéricos	Seu agente `code-reviewer` , que aplica o seu `CLAUDE.md` e o seu `CODIGO.md`
`/security-review`	Skill do produto: revisa as mudanças pendentes da branch	Seu agente `security-reviewer` , que faz triagem entre dez módulos (OWASP, LGPD, secrets, supply chain) e devolve princípios para a constituição

Os nomes diferem por uma letra (`review` contra `review**er**`), e os dois mundos não se atropelam: os seus são invocados pelo fluxo, pela ferramenta de agentes; os do produto, só quando

você digita a barra. O risco não é técnico, é de dedo: digitar `/security-review` esperando a auditoria completa e receber uma revisão de diff.

Nada impede usar as duas de propósito. A do produto é uma boa segunda opinião antes de um PR grande, justamente porque aplica critérios diferentes dos seus.

3. O mapa do fluxo

```
BLOCO A - abertura do projeto (uma vez)
   A1 Briefing e diagnostico
   A2 Documentos-base: ROADMAP, CHECKLIST, ARCHITECTURE
   A3 Provisionamento: GitHub/Linear, .github/, e a constituicao do projeto

BLOCO B - ciclo de feature (repete a cada etapa)
   Especificar -> Projetar -> Tarefas -> Plano -> Executar
   Portao 1  audit --ci --feature <f> = 0   (a maquina prova)
   Portao 2  code-reviewer -> docstrings-manager -> security (se sensivel)
   Portao 3  PR, merge, CHECKLIST atualizado

BLOCO C - fechamento de milestone
   C1 Revisao de seguranca
   C2 Documentacao (tecnica, executiva, manual, apresentacao)
   C3 Release: CHANGELOG, tag, milestone fechada
```

Uma **Etapa** do CHECKLIST equivale a uma issue no board, a uma feature em `.spec/features/` e a uma branch. Esse é o elo que amarra tudo: se você entender só uma coisa deste manual, entenda essa.

4. Antes de começar

Confira uma vez que o ambiente tem o necessário:

```
node --version     # o motor de auditoria roda nele
git --version
gh auth status     # so se voce for usar board no GitHub
```

Atalho que economiza digitação (o motor mora dentro da skill, sem instalação):

```
alias fspec='node ~/.claude/skills/fuzkrigo-spec-driven/scripts/fuzkrigo-spec.mjs'
```

Neste manual, `fspec` significa esse comando. Todos eles rodam **a partir da raiz do projeto**.

5. Abrindo um projeto (Bloco A)

Este bloco se aplica ao seu caso?

Nem todo código nasce projeto. O critério é o destino do que você está construindo:

Se o que você está fazendo...	Bloco A
vai para o portfólio	Obrigatório
tem usuário final, mesmo que seja só você usando todo dia	Obrigatório
vai ganhar release, tag ou versão	Obrigatório
é experimento ou prova de conceito descartável	Dispensável
é script de uso próprio, automação pessoal	Dispensável

Dispensável não é sem rigor. Você pula a camada de gestão (roadmap, board, documentos de comunicação, release) e vai direto para o Bloco B: a feature continua sendo especificada e o gate continua valendo. É diferente do `#tarefa`, que desliga tudo, inclusive a prova.

Se um experimento virar projeto de verdade, o Bloco A é feito retroativamente, antes do primeiro release. O sinal de que chegou a hora é um destes três: apareceu um segundo usuário, você quer marcar uma versão, ou quer mostrar o trabalho.

O passo a passo

Você diz, em linguagem normal:

> "Quero começar um projeto novo: um app para controlar minhas leituras."

A skill `iniciar-projeto` entra sozinha. O que acontece:

A1 — Briefing. Você é entrevistado: escopo, público, requisitos, o que está fora. Também são levantadas as **restrições inegociáveis** — regras que o sistema nunca pode violar ("nota de um aluno nunca aparece para outro", "senha nunca vai para log"). Guarde-as: elas viram lei em A3.

Nenhuma linha de código é escrita aqui. Se você pedir código, será interrompido — de propósito.

A2 — Documentos-base. São gerados:

- `docs/ROADMAP.md` — o plano vivo: problema, decisões de arquitetura, riscos, milestones.
- `docs/CHECKLIST.md` — o índice de etapas. Não lista tarefas miúdas; lista Milestones, Fases e Etapas, cada uma apontando para a pasta da sua feature.
- `docs/ARCHITECTURE.md` — diagramas Mermaid, gerado pelo agente `project-documentation`.

Assim fica uma etapa no CHECKLIST:

```
### Fase 1 — Autenticação
- [x] E1: Login por e-mail — `.spec/features/login-email/` (auditada)
- [ ] E2: Recuperação de senha — `.spec/features/recuperacao-senha/` (em-implementacao)
- [ ] E3: Editar perfil — spec ainda não criada
```

A3 — Provisionamento e constituição. O agente `project-manager` cria repositório, labels, board, milestones e as issues (uma por etapa). E o projeto ganha sua constituição:

```
fspec init                      # ou: fspec init --preset lgpd-educacao
```

Isso cria `.spec/` e `.spec/constituicao.md`. Agora traduza as restrições de A1 em princípios que a máquina verifica:

```
## P-002 [DEVE] Senha nunca aparece em log
- verificação(proibido): `log.*(senha|password)` em `src/**`
```

Um princípio `[DEVE]` sem verificação executável é acusado pelo próprio audit. Aqui, regra sem dente não existe.

Configure a prova antes de sair de A3. Sem isso o `verify` lê zero testes:

Sua stack	testCommand	reporter
Node	`node --test --test-reporter=tap`	`tap`
pytest	`pytest --junitxml=.spec/verification/raw.xml`	`junit-xml`
Vitest	`npx vitest run --reporter=json --outputFile=.spec/verification/raw.json`	`vitest-json`
Jest	`npx jest --json`	`jest-json`

Reporters que gravam arquivo exigem também `reporterOutputFile` no mesmo caminho. Tudo isso fica em `onpspec.config.json`, na raiz.

6. Desenvolvendo uma feature (Bloco B)

Este é o ciclo que você vai repetir. Exemplo: a etapa E2, "Recuperação de senha".

Passo 1 — Especificar

```
fspec new recuperacao-senha
```

Cria `.spec/features/recuperacao-senha/` com `spec.md` e `tasks.md`. Antes de escrever, o contexto do projeto é lido (ROADMAP e a etapa no CHECKLIST) e as lições já confirmadas:

```
fspec licoes list
```

No `spec.md` você escreve **histórias** e, para cada uma, **critérios de aceite** observáveis:

```
### US-002 — Recuperar acesso por e-mail

Como usuário, quero redefinir minha senha, para voltar a acessar a conta.

#### AC-003 — Link de redefinição expira em 30 minutos

- **Dado** um link de redefinição gerado há mais de 30 minutos
- **Quando** o usuário abre esse link
- **Então** vê a mensagem de link expirado (e nenhuma senha é alterada)
```

Regra de ouro do critério: se um teste não consegue checar, não é critério. "Deve ser seguro" não serve; "senha com menos de 8 caracteres é rejeitada" serve.

Duas seções são obrigatórias, e a ausência delas é erro:

- **Suposições (ASM-xxx)** — você preencheu uma lacuna com um palpite e seguiu.
- **Perguntas em aberto (Q-xxx)** — você parou porque falta informação.

Não tem nenhuma? Escreva "Nenhuma." e desconfie de si mesmo.

Passo 2 — Tarefas

No `tasks.md`, quebre em tarefas com os arquivos que cada uma toca:

```
## T-004 — Gerar e validar token de redefinição [pendente]

- Refs: US-002, AC-003
- Arquivos: src/auth/token.py, src/auth/rotas.py
```

O campo `Arquivos` decide o que pode rodar em paralelo. Capriche nele.

Passo 3 — O teste nasce falhando

```
fspec scaffold recuperacao-senha
```

Gera o esqueleto de teste para cada critério sem teste — todos **falhando**. Essa é a definição de pronto, executável desde o primeiro minuto.

Em Python, a etiqueta vive em dois lugares e os dois importam:

```
def test_ac_003():
    """AC-003: Link de redefinicao expira em 30 minutos @spec:AC-003"""
```

O **nome** é o que o runner reporta (é por ele que a prova é casada); a **docstring** é o que o audit lê para saber que o critério tem teste. Renomear a função quebra a prova; apagar a docstring faz o critério parecer sem teste.

Passo 4 — Implementar

Escreva o código até os testes passarem. Uma tarefa, um commit. Marque `[concluida]` só com prova.

Se houver duas ou mais tarefas pendentes, dá para paralelizar:

```
fspec plano recuperacao-senha     # agrupa tarefas de arquivos disjuntos em faixas
fspec painel recuperacao-senha    # painel ao vivo no navegador, com botao de executar
```

Cada faixa roda em um worktree próprio, com contexto limpo.

Passo 5 — Portão 1: a máquina decide

```
fspec verify recuperacao-senha                # roda os testes e grava a prova
fspec audit --ci --feature recuperacao-senha  # o veredito desta feature
```

O `--feature` limita o veredito à feature da vez: a spec da próxima etapa, ainda em rascunho, não reprova esta. O que é do **projeto** — princípio da constituição violado, código órfão — continua contando, porque esconder isso transformaria o filtro numa porta dos fundos.

Antes de fechar uma milestone ou lançar qualquer coisa, rode **sem** o filtro:

```
fspec audit --ci                 # o veredito do projeto inteiro
```

Um relatório filtrado sempre avisa, em amarelo, que o escopo estava reduzido. Se você errar o nome da feature, o comando falha com código 2 e lista as existentes — nunca sai 0 por engano.

Teste pulado **não** conta como prova — o motor recusa e o audit acusa. Se o audit sair 1, ele diz exatamente o que falta. Corrija e repita, no máximo três vezes; persistindo, pare e reavalie o problema em vez de insistir.

Passo 6 — Portão 2: qualidade

Com o audit em zero, entram os especialistas, nesta ordem:

1. `code-reviewer` — SOLID, DRY, KISS, tamanho de função e arquivo, tratamento de erro.
2. `docstrings-manager` — documenta o que ficou público, sem tocar em lógica.
3. `security-reviewer` — só se a feature toca autenticação, permissão, dinheiro ou dado pessoal.

O portão é dimensionado ao tamanho da feature: em feature pequena (até três arquivos, sem lógica de negócio nova) o `code-reviewer` é opcional e o `docstrings-manager` só entra se nasceu algo público. O `security-reviewer` não segue essa regra — o gatilho dele é o que a feature toca, não o tamanho dela.

Depois do `docstrings-manager`, rode o audit de novo. Esse agente edita arquivos, inclusive os de teste; se um título anotado ou um nome `test_ac_001` mudar, a prova daquele critério some. O audit devolve isso na hora, em vez de você descobrir depois.

Achado de segurança que você aceitar vira **princípio na constituição**. É assim que o mesmo erro não volta: da próxima vez, o audit acusa sozinho. O agente não escreve nada — ele devolve o bloco do princípio pronto no relatório, e quem cola em `.spec/constituicao.md` é o Claude principal, depois da sua aprovação.

Passo 7 — Portão 3: entrega

PR seguindo `~/.claude/references/git-conventions.md`. Se a feature rodou em faixas paralelas, o histórico sai por squash:

```
gh pr merge --squash --delete-branch \
  --subject "feat(auth): adicionar recuperacao de senha (T-004, T-005)" \
  --body "Closes #14"
```

Depois, marque a etapa no CHECKLIST e registre o aprendizado, se houver:

```
fspec licoes sugerir              # o motor aponta onde vale uma licao
```

Lição só entra se estiver ancorada em uma falha real registrada. Caminho limpo não gera lição, e isso é o resultado certo — não uma omissão.

7. Fechando uma milestone (Bloco C)

C1 — Segurança. O `security-reviewer` faz a varredura completa. Itens críticos são resolvidos antes de qualquer outra coisa: não se documenta nem se lança o que tem falha aberta.

C2 — Documentação. O `project-documentation` gera ou atualiza a documentação técnica, o resumo executivo, o manual e a apresentação. Esses documentos descrevem o que **existe** — por isso vêm depois da entrega, não antes.

C3 — Release. O `project-manager` monta CHANGELOG, texto do release e os comandos Git. Antes, ele confere que todas as features estão `auditada` — issue fechada não é prova. A execução do Git é sempre uma escolha sua: você digita ou pede para executarem.

8. Quando você não quer nada disso

Comece a mensagem com `#tarefa`. Isso desliga as duas skills, o TDD e a regra de bloqueio pelo resto da sessão. Serve para: corrigir um bug pequeno, mexer em configuração, escrever um script descartável.

Continua valendo, sempre: não inventar API, não expor segredo, não engolir erro, alertar risco de segurança, PT-BR na conversa e inglês no código.

Para sair: comece uma conversa nova, ou diga "modo normal".

Não confunda com projeto dispensável (seção 5). São dois níveis diferentes de desligamento:

	O que desliga	O que sobra
Projeto dispensável	A camada de gestão: roadmap, board, documentos, release	Especificação, testes, audit — o rigor técnico inteiro
`#tarefa`	Tudo: as duas skills, o TDD, a regra de bloqueio	Só o piso: anti-alucinação, segredos, tratamento de erro, idioma

Use `#tarefa` para o que é pontual e não vai crescer. Use projeto dispensável para o que é pequeno mas continua sendo software.

9. Cheat sheet

```
# projeto
fspec init [--preset base|lgpd-educacao]   # cria .spec/ e a constituicao
fspec status                               # onde cada feature esta
fspec assumptions                          # o que estamos assumindo

# feature
fspec new <feature>                        # cria spec.md e tasks.md
fspec scaffold <feature>                   # gera os testes que falham
fspec tarefa <feature> <T-xxx> <status>    # pendente | em-andamento | concluida
fspec plano <feature>                      # agrupa tarefas em faixas paralelas
fspec painel <feature>                     # acompanhamento ao vivo no navegador

# prova
fspec verify <feature>                     # roda os testes, grava a prova
fspec audit                                # relatorio legivel (projeto inteiro)
fspec audit --ci --feature <feature>       # o gate da feature: exit 0 ou 1
fspec audit --ci                           # o gate do projeto (CI e release)
fspec audit --md relatorio.md              # salva o relatorio

# aprendizado
fspec licoes list | sugerir | add | status
```

10. O audit acusou algo. E agora?

Os mais comuns:

Mensagem	Significa	O que fazer
critério de aceite sem teste	Requisito sem prova nenhuma	`fspec scaffold` e escreva o teste
critério de aceite sem prova	Existe teste, mas nunca passou (ou foi pulado)	Rode `verify` ; skip não é prova
teste órfão	O teste aponta para um critério que sumiu	A spec mudou: atualize o teste
tarefa concluída sem prova	Você marcou `[concluida]` sem o critério provado	Verifique de novo, ou reabra a tarefa
suposição em aberto	A feature quer fechar com um palpite não confirmado	Confirme com quem decide, ou invalide
princípio violado	O código quebrou a constituição	Conserte o código. Nunca o princípio
verificação não olha nenhum arquivo	O glob do princípio não casa nada	Corrija o glob: a regra estava inerte
código órfão	Existe código que nenhuma tarefa mapeia	Mapeie na tarefa, ou questione por que ele existe
prova fraca	A prova veio só do código de saída global	Configure um reporter com granularidade

11. Glossário

Código	Nome	O que é
US-xxx	História de usuário	Quem precisa, do que precisa e por quê
AC-xxx	Critério de aceite	Resultado observável que um teste consegue checar
T-xxx	Tarefa	Passo de implementação, com arquivos declarados

Código	Nome	O que é
ASM-xxx	Suposição	Lacuna preenchida com palpite, ainda sem confirmação
Q-xxx	Pergunta em aberto	Decisão que falta a quem é dono do produto
P-xxx	Princípio	Restrição inegociável, com verificação executável
DoD	Definição de pronto	O conjunto de critérios com prova gravada

Na conversa, use o nome por extenso. O código vai entre parênteses quando precisar.

12. Armadilhas conhecidas

Coisas que vão te morder e não são culpa sua:

1. **O audit sem filtro é do projeto inteiro.** No portão de uma feature, use `--feature <nome>` ; sem ele, a spec da próxima etapa ainda em rascunho reprova a feature que você acabou de terminar. E vale o contrário: não use `--feature` para decidir se o projeto está pronto para release, porque aí ele esconde exatamente o que você precisa ver. Boa prática que continua valendo: uma spec viva por vez.
2. `node --test` **sozinho não gera prova.** Do Node 20 em diante o formato padrão mudou; sem `--test-reporter=tap` o verify lê zero testes e reprova tudo, com a suíte inteira verde.
3. **Nunca renomeie um teste com etiqueta** (`@spec:` ou `test_ac_001`). A prova é casada por ela.
4. **Skip não é prova.** `@pytest.mark.skip` ou `test.skip` fazem o critério continuar sem prova, por mais verde que o terminal pareça.
5. **Glob de princípio que não casa arquivo** vira regra inerte. O audit avisa; não ignore.

13. Mantendo o ecossistema

A configuração vive em dois lugares que precisam andar juntos:

- `~/.claude/` — o que o Claude Code realmente carrega.
- Este repositório — o backup versionado.

Mudou um, espelhe no outro. Isso não é burocracia: os dois já saíram de sincronia uma vez, e a instalação de uma status line chegou a sobrescrever o `settings.json` global, deixando os dois hooks desligados por dias sem ninguém notar.

Se você atualizar a skill `fuzkrigo-spec-driven` a partir do projeto de origem, releia a lista de **patches locais** no `README.md` : são dois conjuntos de ajustes no motor — o suporte a Python e o audit com escopo de feature. As duas suítes que os protegem rodam assim:

```
cd ~/.claude/skills/fuzkrigo-spec-driven/scripts
node --test test/ponte-python.test.mjs test/audit-por-feature.test.mjs
```

São 18 casos. Se algum falhar depois de uma atualização, um patch se perdeu — o `README.md` diz qual e onde.